Code3: Manual Backpropagation

One iteration and verification with PyTorch

Scope. Calculate the gradients at the notebook's initial weights, using one input–target pair and one backward pass, before any weight update. Vectors are row vectors; $(W_\ell)_{ij}$ connects input i to output j. Both layers have no bias.

1. Initial values and forward pass

The notebook specifies

$$X = \begin{bmatrix} 1 & 2 \end{bmatrix}, \quad T = \begin{bmatrix} 0 & 0 \end{bmatrix}, \quad W_1 = \begin{bmatrix} 2 & 1 \\ 0 & 1 \end{bmatrix}, \quad W_2 = \begin{bmatrix} 1 & 2 \\ 1 & 2 \end{bmatrix}.$$

The hidden-layer preactivation and activation are

$$\begin{aligned} S_2 = XW_1 &= \begin{bmatrix} 1(2) + 2(0) & 1(1) + 2(1) \end{bmatrix} = \begin{bmatrix} 2 & 3 \end{bmatrix}, \\ Y_2 = \mathrm{ReLU}(S_2) &= \begin{bmatrix} 2 & 3 \end{bmatrix}. \end{aligned}$$

The output-layer preactivation and activation are

$$\begin{aligned} S_3 = Y_2W_2 &= \begin{bmatrix} 2(1) + 3(1) & 2(2) + 3(2) \end{bmatrix} = \begin{bmatrix} 5 & 10 \end{bmatrix}, \\ Y_3 = \mathrm{ReLU}(S_3) &= \begin{bmatrix} 5 & 10 \end{bmatrix}. \end{aligned}$$

All four preactivations are positive. Hence every ReLU derivative used in this iteration is 1.

2. Mean squared error and output error signal

The default `nn.MSELoss()` averages over the two output elements [2]. Therefore

$$L = \frac{(y_{3,1} - t_1)^2 + (y_{3,2} - t_2)^2}{2} = \frac{5^2 + 10^2}{2} = 62.5.$$

For each output j,

$$\frac{\partial L}{\partial y_{3,j}} = \frac{2}{2}(y_{3,j} - t_j), \qquad \delta_3 = \frac{\partial L}{\partial S_3} = \begin{bmatrix} 5 & 10 \end{bmatrix} \odot \begin{bmatrix} 1 & 1 \end{bmatrix} = \begin{bmatrix} 5 & 10 \end{bmatrix}.$$

Here $\odot$ denotes elementwise multiplication.

3. Gradient of the second-layer weights

Since $s_{3,j} = \sum_i y_{2,i}(W_2)_{ij}$, the chain rule gives

$$\frac{\partial L}{\partial (W_2)_{ij}} = y_{2,i}\delta_{3,j}.$$

Thus all four derivatives are the entries of the outer product

$$\boxed{\nabla_{W_2} L = Y_2^{\mathsf{T}} \delta_3 = \begin{bmatrix} 2 \cdot 5 & 2 \cdot 10 \\ 3 \cdot 5 & 3 \cdot 10 \end{bmatrix} = \begin{bmatrix} 10 & 20 \\ 15 & 30 \end{bmatrix}.}$$

4. Hidden error signal and gradient of the first-layer weights

Each hidden activation affects both outputs. Applying the chain rule,

$$\frac{\partial L}{\partial y_{2,i}} = \sum_j \delta_{3,j}(W_2)_{ij}, \qquad \frac{\partial L}{\partial Y_2} = \delta_3 W_2^\mathsf{T}.$$

In numbers,

$$\delta_3 W_2^\mathsf{T} = \begin{bmatrix}5 & 10\end{bmatrix}\begin{bmatrix}1 & 1\\ 2 & 2\end{bmatrix} = \begin{bmatrix}5(1)+10(2) & 5(1)+10(2)\end{bmatrix} = \begin{bmatrix}25 & 25\end{bmatrix}.$$

Both hidden ReLUs have derivative 1, so

$$\delta_2 = \frac{\partial L}{\partial S_2} = (\delta_3 W_2^\mathsf{T}) \odot \mathrm{ReLU}'(S_2) = \begin{bmatrix}25 & 25\end{bmatrix}.$$

Since $s_{2,j} = \sum_i x_i (W_1)_{ij}$,

$$\frac{\partial L}{\partial (W_1)_{ij}} = x_i \delta_{2,j}, \qquad \boxed{\nabla_{W_1} L = X^\mathsf{T}\delta_2 = \begin{bmatrix}1\cdot 25 & 1\cdot 25\\ 2\cdot 25 & 2\cdot 25\end{bmatrix} = \begin{bmatrix}25 & 25\\ 50 & 50\end{bmatrix}}.$$

5. Verification and the `nn.Linear` transpose

The notebook's direct `torch.matmul` implementation prints

$$\texttt{W1.grad} = \begin{bmatrix}25 & 25\\ 50 & 50\end{bmatrix}, \qquad \texttt{W2.grad} = \begin{bmatrix}10 & 20\\ 15 & 30\end{bmatrix}.$$

These exactly match the manual derivatives above.

PyTorch's `nn.Linear` computes $xA^\mathsf{T} + b$ and stores A with shape (output features, input features) [1]. The notebook sets

$$A_1 = W_1^\mathsf{T} = \begin{bmatrix}2 & 0\\ 1 & 1\end{bmatrix}, \qquad A_2 = W_2^\mathsf{T} = \begin{bmatrix}1 & 1\\ 2 & 2\end{bmatrix}.$$

Thus $XA_1^\mathsf{T} = XW_1$ and $Y_2 A_2^\mathsf{T} = Y_2 W_2$. Since $(A_\ell)_{ji} = (W_\ell)_{ij}$, their parameter gradients obey

$$\nabla_{A_\ell} L = (\nabla_{W_\ell} L)^\mathsf{T}.$$

The executed notebook confirms

$$\texttt{fc1.weight.grad} = \begin{bmatrix}25 & 50\\ 25 & 50\end{bmatrix}, \qquad \texttt{fc2.weight.grad} = \begin{bmatrix}10 & 15\\ 20 & 30\end{bmatrix}.$$

The verification cell checks both direct gradients and both transposed `nn.Linear` gradients against the manual matrices. All four checks pass with maximum absolute error 0. Both implementations give $Y_3 = [5, 10]$ and $L = 62.5$ before the first update.

References

[1] PyTorch documentation, `torch.nn.Linear`. `https://docs.pytorch.org/docs/2.14/generated/torch.nn.Linear.html`

[2] PyTorch documentation, `torch.nn.MSELoss`. `https://docs.pytorch.org/docs/2.14/generated/torch.nn.MSELoss.html`